

Question-1: A password contains n positions. For every position, you can place: ABC . But the password follows these rules:

1. Two Bs cannot appear consecutively.
2. The password cannot contain CCC.
3. If A is placed, the next character must not be B.

Find the number of valid passwords of length n .

Example

$n = 3$

The recursion must remember enough information about the **previous characters** to determine whether the next character is allowed.

Solution:

```
#include<bits/stdc++.h>

using namespace std;

int maximumPasswordsGenerated(int n,string res,char prev,char arr[]){

    //base case

    if(res.size()==n){

        cout<<res<<" ";

        return 1;

    }

    //main logic

    int ways=0;

    for(int i=0;i<3;i++){

        //conditions

        if(prev=='A'&&arr[i]=='B'){
```

```

        continue;

    }

    else if (prev=='B' && arr[i]=='B') {

        continue;

    }

    else
if (res.size() >= 2 && res[res.size()-1] == 'C' && res[res.size()-2] == 'C' && arr[i] == 'C') {

        continue;

    }

    else {

        ways += maximumPasswordsGenerated(n, res + arr[i], arr[i], arr);

    }

}

return ways;

}

int main() {

    int n = 3;

    string res = "";

    char arr[] = {'A', 'B', 'C'};

    char prev = '&';

    cout << maximumPasswordsGenerated(n, res, prev, arr);

    return 0;

}

```

Question-2:

There are three jars labelled A, B, and C, and all are incorrectly labelled.

- Jar A: “Candies”
- Jar B: “Sweets”
- Jar C: “Candies and Sweets” (mixed)

You are allowed to pick only one item at a time from any jar, and candies and sweets can only be identified after picking.

Solution:**Step 1: Start with Jar C (labeled “Candies & Sweets”)**

- All jars are incorrectly labeled.
- So, Jar C cannot be a mixture.
- It must contain only Candies or only Sweets.
- Pick one item from Jar C.
- Suppose you get a candy → Jar C = Candies only

**Step 2: Identify Jar B**

- Jar B is labeled “Sweets”, but labels are wrong.
- So, Jar B cannot be only sweets.
- It also cannot be candies (because Jar C already has candies).
- Therefore, Jar B must be Candies & Sweets (mixture)



Step 3: Identify Jar A

- Only option left is Sweets
- So, Jar A = Sweets

Final Result

- *Jar A contains Sweets.*
- *Jar B contains Candies and Sweets (mixture).*
- *Jar C contains Candies.*

Question-3:

A milkman has 100 cows numbered from 1 to 100. Every cow gives milk according to its number, i.e. i 'th cow gives i litres of milk. Milkman has 10 sons, and he wants to divide his cows among his 10 sons so that every son gets equal amount of milk.

Solution:

As we know that the i 'th cow gives i litres of milk.

So first, let us count the total litres of milk which is obtained from these cows. This can be solved by using the arithmetic progression sum.

We know that the sum of n numbers starting from 1 is always $\text{sum} = \frac{n(n+1)}{2}$.

So the total quantity of milk obtained from 100 cows is total milk = $\frac{100(100+1)}{2} = 5050$ litres.

As the problem says that the division of the cows should be done in a manner that every son gets an amount of milk.

So every son should get $5050/10 = 505$ litres of milk.

Now our main problem is to divide 1 to 100 numbers in such a way that every son should get 10 cows whose numbers sum up to 505.

Example :

Now, let us suppose you have 10 numbers and you have to divide them among 5 people so that each one gets an equal sum of these numbers.

So these 10 numbers sum up to 55, now each of the 5 people will get 11 as a sum, so it's obvious that every person will get two numbers whose sum is 11. So the division would be like this:

First_person ---> 1, 10

Second_person ---> 2, 9

Third_person ---> 3, 8

Fourth_person ---> 4, 7

Fifth_person ---> 5, 6 so each person will get 11 .

Now coming back to the problem, we have 100 numbers we have to divide this in 10 groups so that each son get 505 litre of milk. So,

First_son ---> 1, 2, 3, 4, 5, 96, 97, 98, 99, 100

Second_son ---> 6, 7, 8, 9, 10, 91, 92, 93, 94, 95

Third_son ---> 11, 12, 13, 14, 15, 86, 87, 88, 89, 90

Fourth_son ---> 16, 17, 18, 19, 20, 81, 82, 83, 84, 85

Fifth_son ---> 21, 22, 23, 24, 25, 76, 77, 78, 79, 80

Sixth_son ---> 26, 27, 28, 29, 30, 71, 72, 73, 74, 75

Seventh_son ---> 31, 32, 33, 34, 35, 66, 67, 68, 69, 70

Eighth_son ---> 36, 37, 38, 39, 40, 61, 62, 63, 64, 65

Ninth_son ---> 41, 42, 43, 44, 45, 56, 57, 58, 59, 60

Tenth_son ---> 46, 47, 48, 49, 50, 51, 52, 53, 54, 55

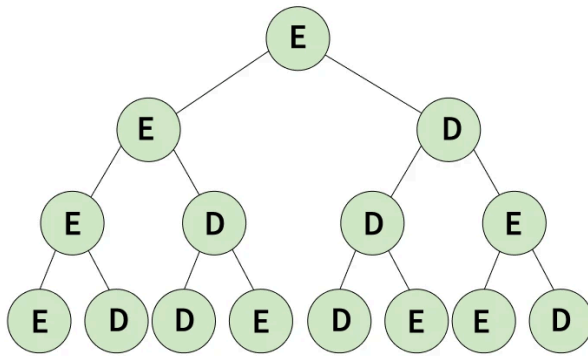
Each son should get equal amount of milk this way.

Question-4:

Consider a special family of **Engineers** and **Doctors** with following rules :

- Everybody has two children.
- First child of an Engineer is an Engineer and second child is a Doctor.
- First child of an Doctor is Doctor and second child is an Engineer.
- All generations of Doctors and Engineers start with Engineer.

The first few levels of the family tree are shown below :



Given the 1-based **level** and **pos** (position from left) of a person in above ancestor tree, return their profession as a string: either "Engineer" or "Doctor".

Examples:

Input: level = 4, pos = 2

Output: Doctor

Explanation: It is shown in the tree given in question.

Input: level = 3, pos = 4

Output: Engineer

Explanation: Already given in the tree in question.

Constraints:

$$1 \leq \text{level} \leq 10^9$$

$$1 \leq \text{pos} \leq 10^9$$

Solution:

```
string profession(int level, int pos) {  
    // code here// Base case  
    if (level == 1)  
        return "Engineer";  
}
```

```
// Go to parent

string parent = profession(level - 1, (pos + 1) / 2);


// First child -> same profession
if (pos % 2 == 1)

    return parent;


// Second child -> opposite profession
if (parent == "Engineer")

    return "Doctor";

else

    return "Engineer";

}
```

Note: Since it's a recursive solution , so it may give TLE/Runtime Error/Segmentation Fault. We did it because we are working with recursion else it has a queue/iterative solution too.